

接下来最合适的工作，是把《套马的汉子》从“有趣的概念”压缩成一个**真正可以开始做的核心原型**。

现阶段不要急着设计完整开放世界、剧情、育种、部落和几十种动物。我们先回答一个最关键的问题：

玩家骑着马，在草原上追逐另一匹野马，最后成功套住它——这 3~5 分钟本身，能不能好玩到让人反复玩？

下面我直接把这一阶段定成《套马的汉子：核心玩法原型 v0.1》。

一、Prototype v0.1 的目标

第一版只验证一个完整体验：

发现野马 → 接近 → 追逐 → 马群逃跑 → 锁定目标 → 抛套索 → 套中 → 拉扯 → 控制目标 → 驯服成功。

整个过程控制在：

3~8 分钟。

玩家玩完以后应该产生三种感受：

- “骑马跑起来很爽。”
- “我刚才差一点就套到了。”
- “再来一次，我肯定能抓到那匹黑马。”

如果能做到这一点，游戏就值得继续。

二、第一张原型地图

不要做开放世界。

先做一张大约：

2km × 2km

的测试草原。

里面只有几个地形元素：

- 大片平缓草原
- 一条浅河
- 一个缓坡
- 一片稀疏树林
- 几块岩石
- 一处围栏牧场

目的不是漂亮，而是测试不同追逐环境。

例如：



野马会根据追逐情况进入不同区域。

玩家自然会发现：

平地追最快，但树林可以把马逼入死角；河流会降低速度；山坡会影响冲刺。

地图开始成为玩法的一部分。

三、玩家控制

建议采用：

第三人称骑乘

镜头稍微靠后。

不是传统“角色坐在移动载具上”。

玩家需要同时控制：

骑手 + 马。

基本操作可以先设计成：

WASD	控制方向
Shift	催马 / 加速
Ctrl	减速
Space	跳跃
鼠标	镜头
右键	准备套索
左键	抛出套索
滚轮	收绳 / 放绳
Q	放弃套索



其中有一个很重要的设计：

镜头方向 ≠ 马的方向

这意味着：

你可以骑马向前奔跑，

同时转头观察左侧马群。

这样才能真正产生“骑手感”。

四、骑马不能像开汽车

这可能是整个项目第一技术难点。

玩家输入：

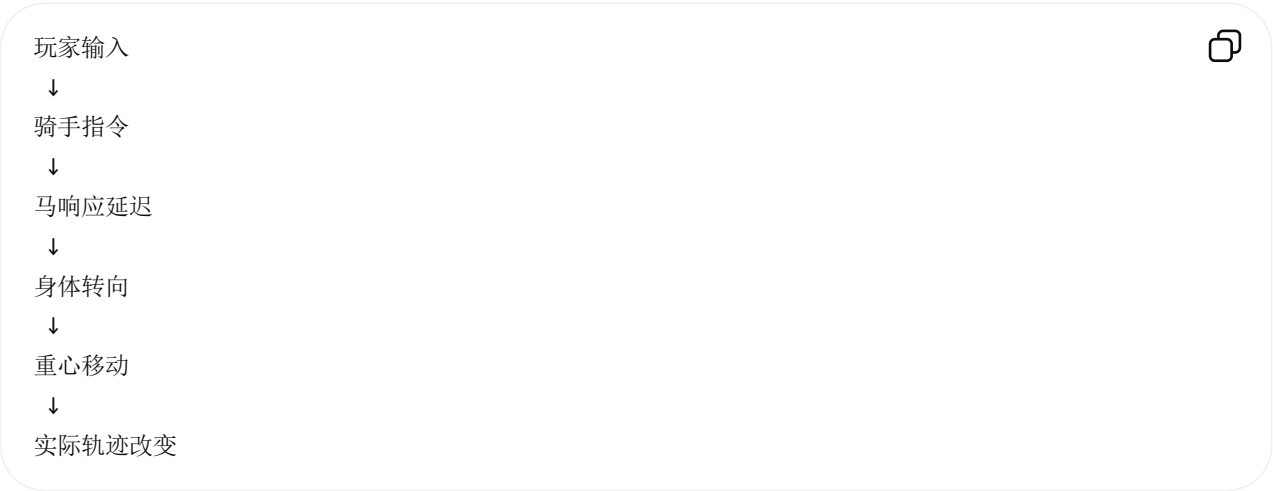
左转

不应该立即：

旋转30°

而应该经过马匹运动系统。

例如：



不同速度下转向能力不同。

例如：

慢速

可以：

- 原地转向
- 小半径移动
- 精确控制

小跑

转向自然。

奔跑

转向半径明显增大。

全速冲刺

突然猛转：

马会失去平衡。

甚至：

摔倒。

这样玩家会开始学习：

不要一直按住Shift。

五、马匹运动状态

Prototype 第一版只需要：

Idle

Walk

Trot

Canter

Gallop

Sprint

Brake

Turn

Jump

Stumble

Fall



其中最重要的是：

Gallop

正常高速追逐。

和：

Sprint

短时间爆发。

Sprint 应该消耗：

Horse Stamina。

于是玩家追马时会产生策略：

我要现在冲刺接近，还是保留体力？

六、不要使用简单的“马匹耐力条”

UI当然可以显示耐力。

但是马本身也要通过动画反馈。

比如：

耐力下降：

- 呼吸变重
- 头部运动增加
- 马蹄节奏改变
- 速度逐渐下降

极度疲劳：

- 不愿继续冲刺
- 转向变慢

最终玩家应该能：

看马判断状态。

而不是一直盯着HUD。

七、野马AI v0.1

第一版不用特别复杂。

但绝对不能：

玩家距离 < 10m
→ 野马直线逃跑



应该做一个简单的：

Threat System

每匹野马会有：

fear
alertness
stamina
confidence
herdAffinity



例如：

玩家距离：

50m。

马可能只是：

抬头。

耳朵朝玩家方向。

30m：

警戒。

20m：

开始移动。

10m：

逃跑。

但这不是固定值。

比如胆小的马：

40m就跑。

大胆的：

15m才跑。

因此玩家第一次就会发现：

不同马真的不一样。

八、马群AI

Prototype建议：

12 ~ 20匹马。

不用每匹独立做复杂决策。

采用：

Herd Leader + Boids

基本规则：

Separation

避免撞其他马

Alignment

跟随群体方向

Cohesion

不要离群太远



再加入：

Threat Avoidance

Terrain Avoidance

Leader Following



于是：

玩家从右侧冲过去。

整个马群：

开始向左偏。

如果玩家继续压迫：

马会整体改变方向。

这就产生非常重要的玩法：

赶马

甚至还没有套索的时候，

玩家已经可以：

操纵马群。

九、追逐不是“追最近的马”

这是非常重要的地方。

玩家应该观察马群。

然后选择目标。

例如：

15匹马里面：

- 白马：速度快
- 黑马：耐力高
- 栗色马：胆小
- 小马：容易捕获
- 马王：极难捕获

当玩家开始追一匹马：

目标会试图：

重新融入马群。

这很重要。

因为马群会成为：

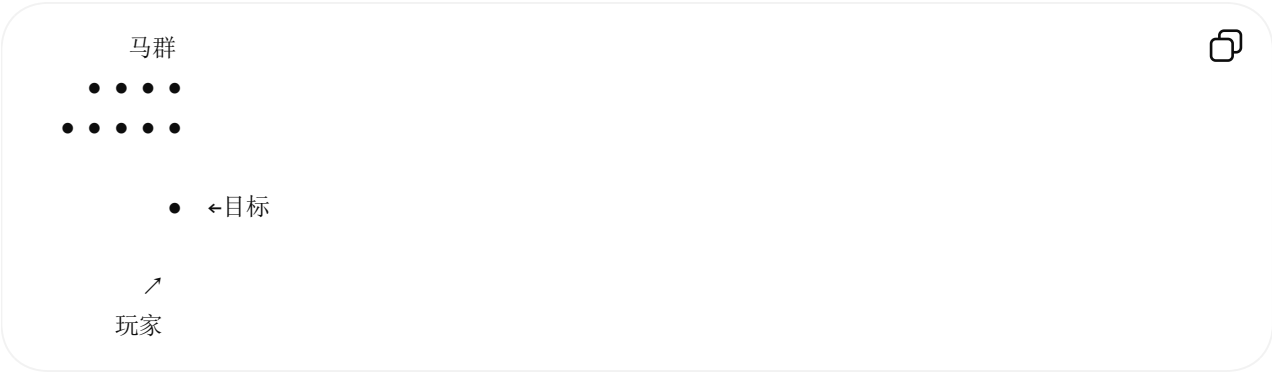
天然屏障。

玩家要把目标：

从马群里切出来。

这就会产生一种类似牧牛的技巧。

例如：



玩家不能从后面一直追。

而是需要：

切线逼迫目标离群。

这会形成很深的技术空间。

十、马群应该“读玩家的位置”

野马判断威胁方向。

例如玩家在：

右侧。

马群会：

向左移动。

于是高手可以通过自己的骑行路线：

控制整个马群方向。

Eventually：

高手追马看起来就像：

牧羊犬驱赶羊群。

而不是：

赛车追车。

这将成为《套马的汉子》的标志性玩法。

十一、套索系统 Prototype

现在进入最重要的部分。

玩家按：

右键。

角色拿起套索。

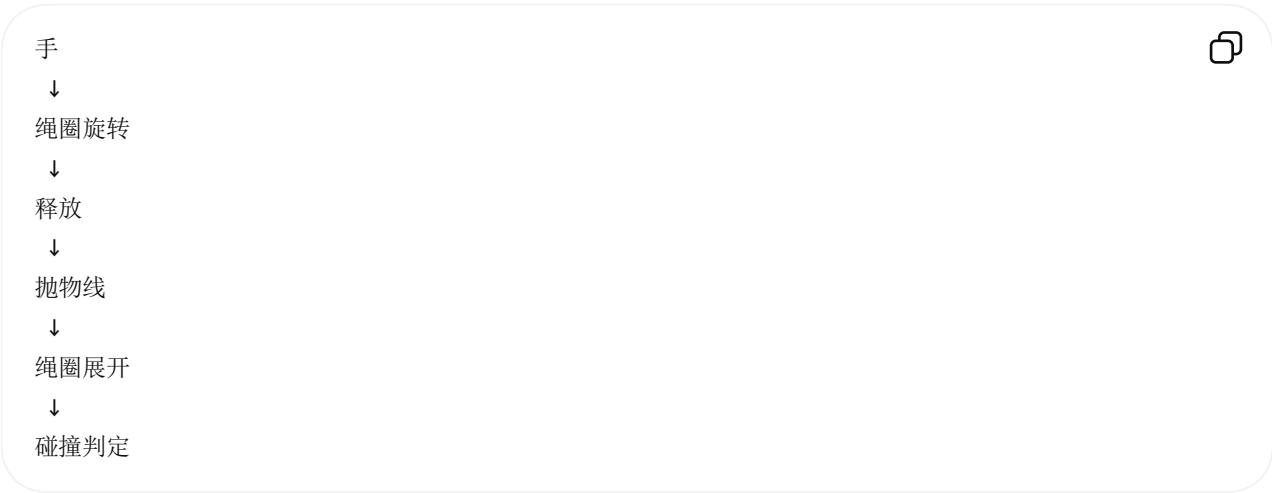
开始旋转。

这里不要做准星射击。

而应该：

套索有真正的运动轨迹

至少视觉和判定上应该模拟：



玩家控制：

旋转时间

旋转越久：

出手越稳定。

但：

马会跑远。

鼠标方向：

控制：

抛掷角度。

马速：

影响：

套索初速度。

于是会出现高手技巧：

高速侧向抛索。

十二、套索不是一个点

最好把它理解成：

一个动态圆环

可以套到目标不同部位。

Prototype 第一版可以简化为：

Head
Neck
Torso
FrontLeg
BackLeg



不同位置效果不同。

例如：

Neck

最佳。

稳定控制。

Body

能控制。

但容易滑脱。

Leg

危险。

目标可能：

摔倒。

这会立即产生一个非常重要的问题：

玩家要不要为了成功，冒险套马腿？

这就是游戏设计。

十三、套中以后，不要进入动画

很多游戏会变成：

套中
↓
播放动画
↓
捕获成功



我们不要。

套中以后：

游戏才真正开始。

立即进入：

Tension Gameplay。

十四、绳索张力

套索有：

CurrentLength
MaxLength
Tension
BreakingStrength
Elasticity



最重要的是：

Tension

可以用简单公式：

Tension =
DistanceForce
+
RelativeVelocity
+
HorsePullForce



UI上不要显示精确数字。

只显示：

松弛
正常
危险
即将断裂



甚至直接通过：

绳索颜色 / 震动 / 声音。

十五、套中后最有意思的行为

假设：

你向东跑。

野马向北跑。

绳子开始拉紧。

玩家有几个选择。

A. 强行拉

迅速减速目标。

但：

容易断绳。

B. 跟着跑

保持适当距离。

慢慢消耗目标体力。

C. 横向施力

改变野马方向。

D. 绕圈

把野马逐渐逼停。

于是：

捕马开始类似钓鱼。

实际上这是一种非常好的参考：

Fishing Gameplay

钓鱼游戏中：

鱼往左跑。

你不能硬收线。

需要：

放线。

鱼累了：

再收。

套马完全可以采用类似思想，

但是：

发生在60km/h的骑马追逐中。

这就很独特。

十六、玩家落马

我认为必须做。

当：

Tension

>

RiderBalance

玩家可能：

被从马上拉下来。

尤其：

目标突然改变方向。

这会产生很多有趣情况。

例如：

你套中一匹超级强壮的野马。

你觉得：

“抓到了！”

下一秒：

它突然反向。

砰。

玩家飞出去。

十七、落马以后不要直接失败

这是另一个很重要的设计。

玩家掉下来。

但是：

手里还抓着绳子。

于是：

马会继续跑。

玩家：

被拖行。

这时：

可以：

松手



或者：



于是就会出现：

最愚蠢却最精彩的玩家行为：

“我能拉住！”

然后被拖300米。

这种事情非常适合传播。

十八、马匹疲劳

野马也有：

Stamina

Stress

Fear



被套住以后：

Stress迅速增加。

然后：

马会：

冲刺。

突然转向。

跳跃。

尥蹶子。

随着时间：

体力下降。

最终：

从：



变成：

抵抗



再变成：

站立



但这还不是：

驯服。

只是：

控制住。

十九、捕获之后的第一个互动

玩家下马。

慢慢靠近。

野马会：

后退。

耳朵动作。

呼吸。

甚至：

踢人。

这里可以设计非常简单的：

Approach System

玩家不能直接冲过去。

需要：

停。

观察。

靠近。

最终：

摸到马。

第一次成功触摸：

游戏可以出现非常轻的反馈：

建立第一次信任。

不要：

捕获成功！
+500 XP

那样会破坏感觉。

二十、Prototype成功结尾

玩家把野马：

牵回营地。

然后：

给它取名字。

这一步非常重要。

因为：

命名 = 建立所有权和情感。

系统显示：

野马

性别：母
年龄：4岁
毛色：黑
性格：警觉 / 独立

速度★★★★☆
耐力★★★★★
力量★★★☆☆
敏捷★★★★☆

信任：



然后：

玩家输入名字。

例如：

黑风

这一刻：

第一次游戏循环完成。

二十一、第一局完整体验

我甚至建议原型第一局：

00:00

玩家站在营地。

NPC：

“山坡那边有一群野马。”

00:30

玩家骑马出发。

01:30

远处出现马群。

02:00

慢慢靠近。

02:30

马会警觉。

03:00

开始逃跑。

音乐开始。

03:30

玩家第一次抛套。

失败。

04:00

第二次。

又失败。

04:30

终于套中。

04:35

野马暴冲。

05:00

开始拉扯。

06:00

玩家逐渐控制住。

06:30

下马靠近。

07:00

牵回营地。

08:00

给马取名字。

这个：

8分钟体验

就应该成为：

整个游戏开发的第一块基石。

二十二、Prototype第一版暂时完全不要做什么

非常重要。

现在先不要碰：

- 建造系统
- 部落系统
- 经济系统
- 交易系统
- 育种
- 冬天
- 天气
- 狼
- 剧情
- RPG属性
- 装备系统
- 技能树
- 任务系统
- 大地图
- 多人游戏

甚至：

先不要做马厩。

因为现在只有一个问题：

骑马追马到底有没有意思？

二十三、技术结构应该从一开始留好扩展空间

即使Prototype简单，我仍建议把马设计成：

HorseEntity

|

|— HorseIdentity

| |— name

| |— sex

| |— age

| |— coat

|

|— HorsePhysical

| |— speed

| |— acceleration

| |— stamina

| |— strength

| |— agility

|

|— HorsePersonality

| |— fear

| |— aggression

| |— curiosity

| |— independence

|

|— HorseMovement

|

|— HorseBrain

|

|— HorseHerd

|

|— HorseRelationship

Prototype可以只实际使用：

speed

stamina

fear

agility

但架构先留出来。

二十四、AI架构

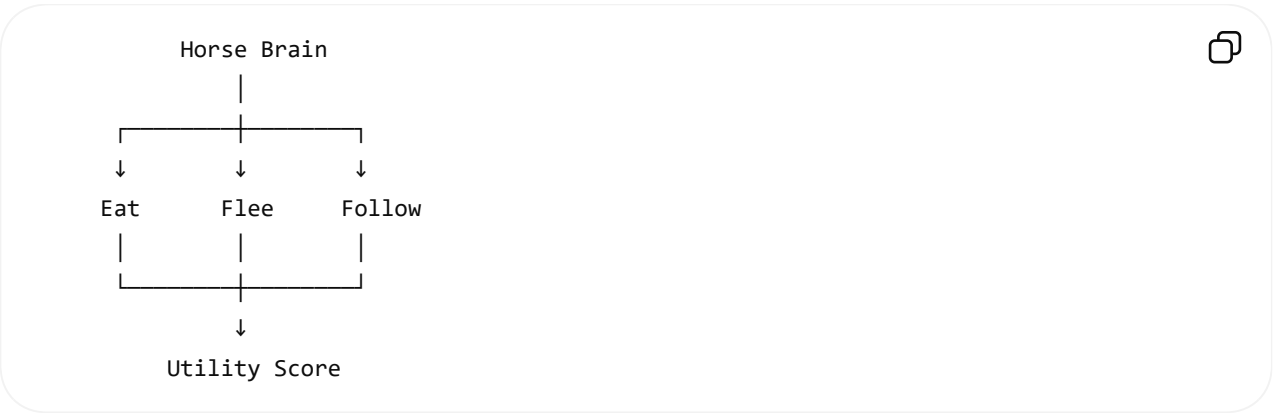
马AI最好不要写成：

几十个if。

我推荐：

Utility AI + State Machine

例如：



例如：

```
FleeScore =  
Threat × Fear × DistanceFactor
```

```
FollowHerdScore =  
DistanceFromHerd × HerdAffinity
```

```
RestScore =  
1 - Stamina
```

然后：

最高行为胜出。

以后增加：

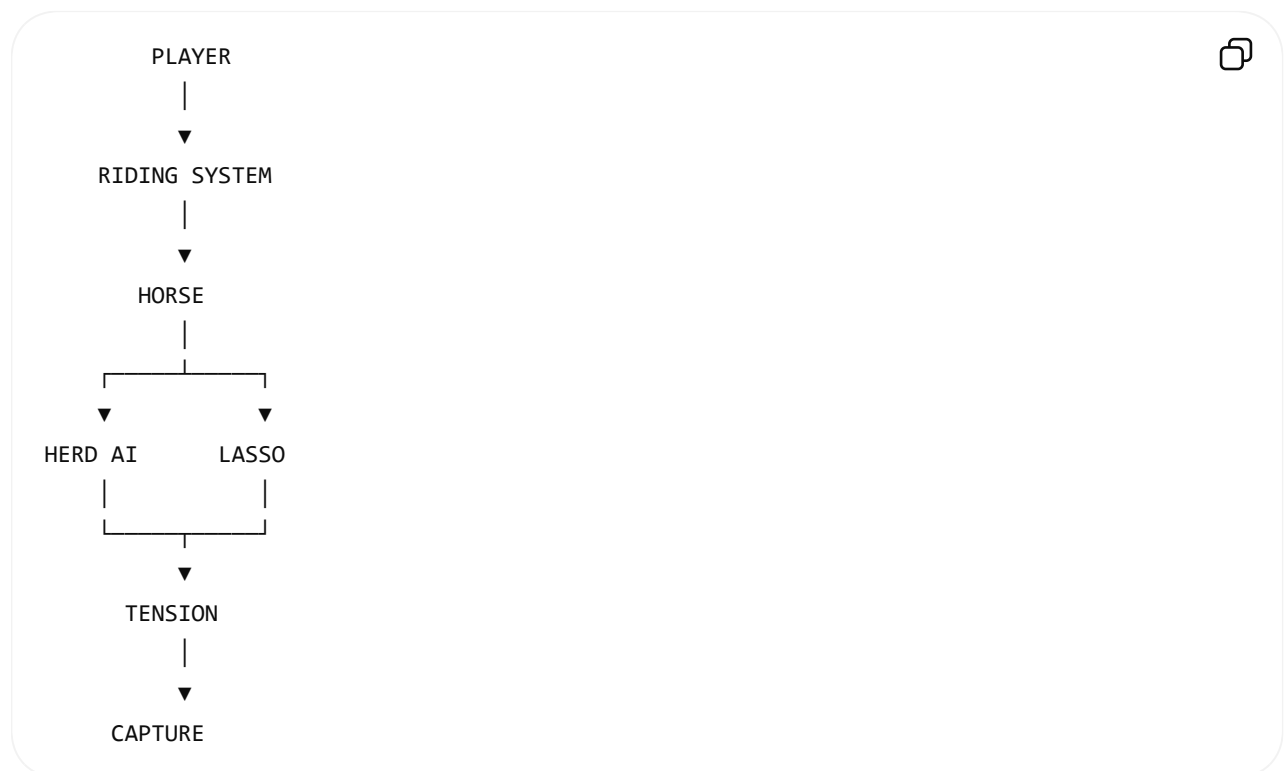
- 吃草
- 喝水
- 寻找幼驹

- 寻找配偶
- 对抗马王
- 躲避狼

都不需要推翻AI。

二十五、最关键的系统关系

最终Prototype其实只有四套核心系统：



如果这四个系统互相作用得好，

游戏就开始成立。

二十六、我建议加入一个隐藏的核心设计：马的“意图”

马不是：

输入 → 动画。

马应该有一个：

Desired Movement

例如：

野马AI：

我要向西逃。

实际移动系统：

但是我正在高速向北跑，所以需要先减小北向速度，再逐渐转向西。

这样：

AI决定意图。

物理系统决定能不能做到。

这是非常重要的架构原则。

否则马就会像机器人。

二十七、Camera非常关键

骑马游戏很容易：

镜头抖。

或者：

感觉很慢。

需要专门设计：

Horse Camera

速度越快：

FOV：

70°

→

78°



镜头稍微：

后移。

轻微：

上下运动。

但千万不要剧烈晃动。

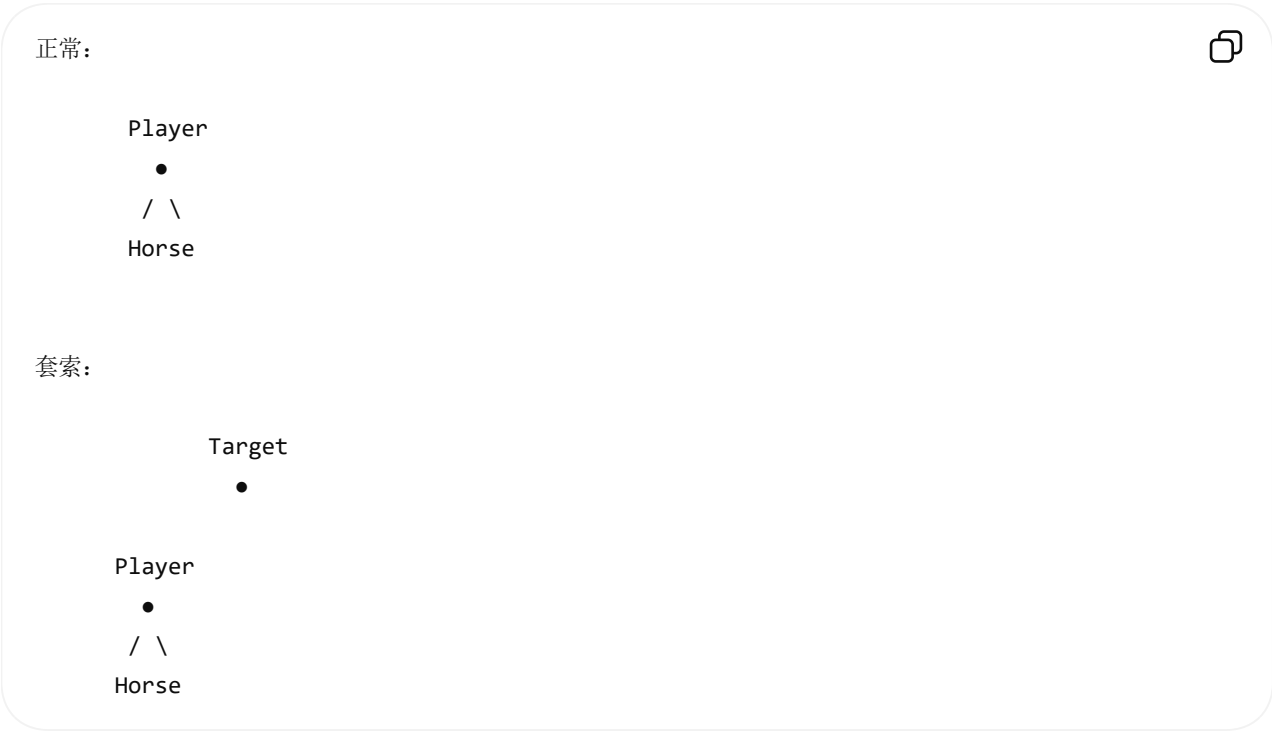
另外当玩家：

准备套索时，

镜头可以：

向右侧偏移。

例如：



根据：

体力。

绳索

- 摇动
- 飞行
- 拉紧
- 吱嘎
- 断裂

风

随着速度增强。

追逐时：

马蹄 + 风 + 呼吸

本身就能产生：

非常强的速度感。

二十九、第一阶段开发顺序

我建议严格按这个顺序。

Phase 0A —— 一匹可以骑的马

只做：

Player

Horse

Camera

Movement

验收标准：

在一块空草地上骑10分钟仍然觉得舒服。

Phase 0B —— 野马

增加：

WildHorse
Fear
Flee
Stamina



验收：

玩家靠近。

马会自然逃跑。

Phase 0C —— 马群

加入：

Herd
Leader
Separation
Alignment
Cohesion



验收：

20匹马能像一个群体一样移动。

Phase 0D —— 追逐

加入：

玩家压迫方向。

马群根据：

玩家位置改变方向。

验收：

玩家可以主动把马群：

赶向某个区域。

Phase 0E —— 套索

实现：

Swing
Throw
Collision
Attach
Rope



验收：

抛套索本身有技巧。

Phase 0F —— 张力

加入：

rope tension
horse pulling
rider balance
rope breaking



验收：

套中不代表成功。

Phase 0G —— 捕获

增加：

horse exhaustion
approach
lead
name



完成：

第一条完整玩法循环。

三十、Prototype最终验收测试

不要问：

功能有没有bug。

而要问玩家：

Test 01

你愿不愿意：

连续抓5匹马？

Test 02

你有没有感觉：

“这次失败是我操作不好。”

而不是：

“系统判定有问题。”

Test 03

不同马：

有没有明显差别？

Test 04

成功套中：

有没有爽感？

Test 05

绳子快断的时候：

有没有紧张感？

Test 06

掉下马：

是愤怒，

还是：

“哈哈哈哈我怎么飞了。”

如果是后者：

其实是非常好的信号。

三十一、第一版最重要的三个指标

我甚至会把整个Prototype开发看成三个数字：

Chase Time

玩家愿意追多久？

目标：

> 3分钟仍然不烦。

Retry Desire

失败以后：

还想不想再来？

目标：

非常强。

Story Density

每10分钟：

有没有发生值得玩家讲给别人听的事情？

例如：

“我刚套到一匹马结果它直接把我拖河里去了。”

如果很多：

这个游戏就非常有潜力。

三十二、真正值得追求的效果

我们最终不是要让玩家说：

“这个马的AI做得不错。”

而是：

“那匹黑马特别狡猾。”

也不是：

“这个Physics Rope系统不错。”

而是：

“妈的，绳子差一点断了。”

也不是：

“Herd AI算法效果不错。”

而是：

“我刚冲进去，整个马群突然炸了。”

玩家应该描述故事，而不是描述系统。

这其实可以作为《套马的汉子》非常重要的设计哲学。

三十三、从现在开始，我建议给项目一个内部开发代号

正式中文名依然可以：

《套马的汉子》

英文品牌名我们以后再定。

但开发项目可以暂时叫：

Project STEPPE

因为现在我们真正需要开发的不是“一款庞大的草原游戏”，而是一个非常具体的东西：

STEPPE Prototype — Horse Chase & Lasso Simulation

接下来最合适的阶段，就是进入 **Prototype 技术设计**：把 HorseController、HorseMovementComponent、HorseBrain、HerdManager、LassoController、RopeSimulation、RiderBalance、CaptureSystem 这些模块正式拆出来，再决定使用 Unity、Unreal 还是 Godot，并最终形成一份可以直接交给 Cursor/Codex 开始搭项目的《套马的汉子 Prototype 技术架构与开发任务书 v0.1》。



我赞成直接把技术路线定到 **Unreal Engine 5**。尤其《套马的汉子》的核心恰好落在 UE 擅长的领域：角色动画、Chaos 物理、复杂环境、开放世界、AI、大规模实体以及后续联机。当前 Epic 文档已经是 UE 5.8；World Partition、MassEntity、PCG、StateTree 等也都有现成框架可利用。 Epic Games Dev... +3

下面我直接把项目推进到第一份真正可以指导开发的技术文件。

《套马的汉子 / Project STEPPE》Prototype 技术架构与...



《套马的汉子 / Project STEPPE》 Prototype 技术架构与开发任务书 v0.1

0. 项目定位

项目暂定中文名：

《套马的汉子》

内部开发代号：

Project STEPPE

游戏类型：

第三人称开放世界草原模拟 / 动作追逐 / 马群生态 / 牧业经营

但当前 Prototype 阶段不要开发完整开放世界游戏。

第一阶段唯一目标：

做出“骑马追逐野马 → 从马群中切出目标 → 抛出套索 → 套中 → 拉扯 → 控制 → 带回营地”的完整核心循环。

Prototype 是否成功，只由一件事情判断：

不依赖剧情、奖励、养成和任务，仅仅骑马追一匹野马，玩家愿不愿意重复玩。

1. 技术路线

Engine

推荐：

Unreal Engine 5.8

如果个别 Marketplace 动画、马匹资源或关键插件尚未完整支持 5.8，则允许开发阶段退到一个确认兼容的 5.x 版本，但不要因为素材反过来污染核心架构。

2. 编程策略

采用：

C++ Core + Blueprint Presentation

原则：

C++负责

- Horse locomotion
- Horse attributes
- Horse AI
- Herd simulation
- Threat evaluation
- Lasso state
- Rope gameplay simulation
- Rope tension
- Capture logic
- Rider balance
- Save data
- Debug systems

- 性能关键代码

Blueprint负责

- Animation Blueprint
- Montage
- 摄像机参数调节
- VFX
- SFX
- UI
- Level scripting
- Prototype参数调节
- 快速游戏体验实验

禁止两个极端：

不要全Blueprint。

否则后期马群、AI、物理、网络同步会越来越难管理。

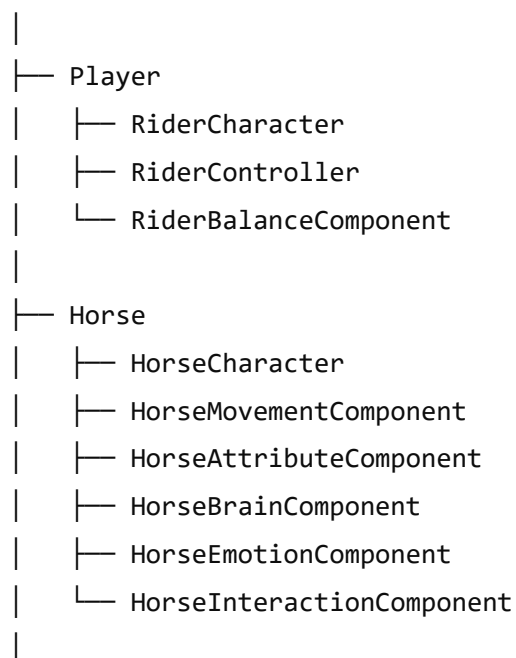
也不要：

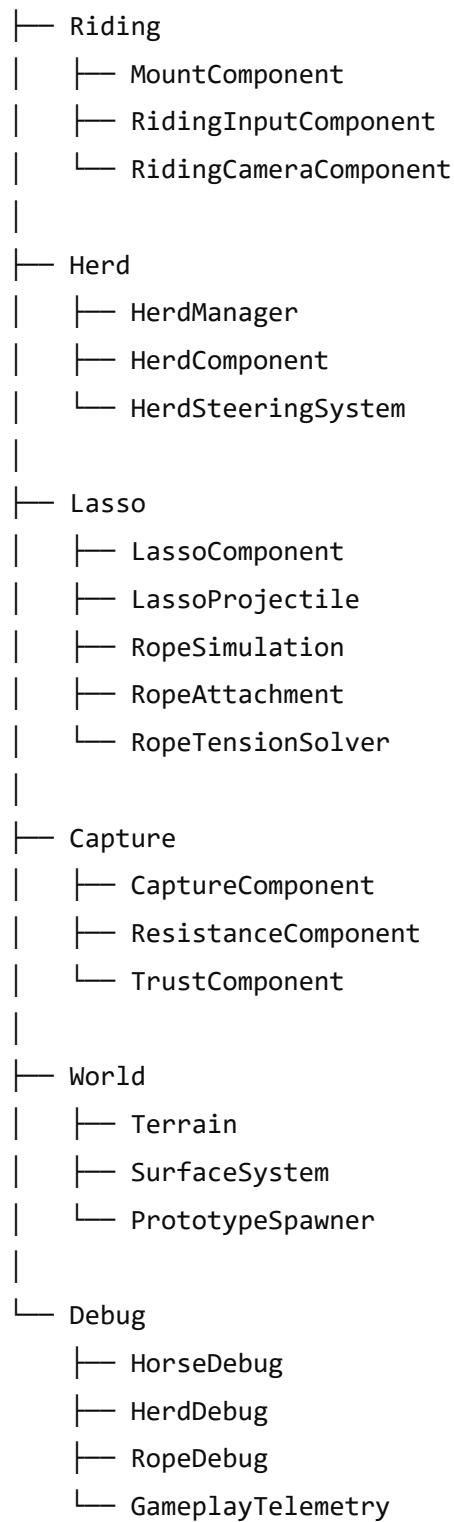
所有东西全部C++。

游戏手感需要非常高频率地调参。

3. Prototype总体结构

Project STEPPE





4. 最重要的架构原则

Horse ≠ Vehicle

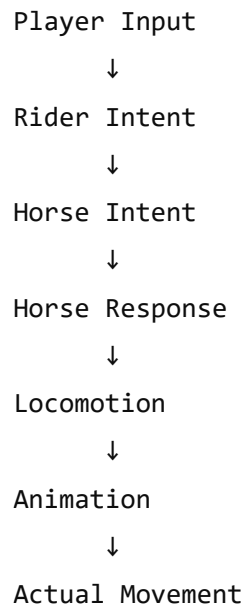
绝对不能把马写成普通Vehicle Pawn。

玩家操作的不是：

Input

→ Velocity

而是：



这层“Horse Intent”非常重要。

玩家是在向动物提出要求，而不是直接控制一个机械装置。

5. AHorseCharacter

建立：

AHorseCharacter

建议继承：

ACharacter

Prototype阶段优先利用成熟 Character Movement 基础能力，再逐渐替换真正需要自定义的运动部分。

基础组件：

CapsuleComponent

SkeletalMesh

HorseMovementComponent

HorseAttributeComponent

HorseBrainComponent

HorseEmotionComponent

HorseInteractionComponent

HerdComponent

LassoTargetComponent

6. HorseAttributeComponent

第一阶段至少支持：

MaxSpeed

Acceleration

Deceleration

TurnRate

Agility

MaxStamina

CurrentStamina

StaminaRecovery

Strength

Fearfulness

Independence

BodyMass

注意：

这些属性绝对不能只是RPG数字。

必须真正进入：

运动 / AI / 套索计算。

例如：

Strength

↓

拉绳力量

Agility

↓

高速转弯能力

Fearfulness

↓

警戒距离

Stamina



逃跑持续时间

这样玩家以后才会真正感觉到：

“这匹马虽然跑得不算最快，但是特别难控制。”

7. Horse Locomotion

Prototype阶段支持：

Idle

Walk

Trot

Canter

Gallop

Sprint

Brake

SharpTurn

Jump

Stumble

Fall

但内部不要硬切状态。

推荐建立连续变量：

DesiredSpeed

CurrentSpeed

DesiredDirection

CurrentDirection

TurnIntent

AccelerationIntent

BrakeIntent

再由运动系统推导 gait。

8. 马的转向

这是第一阶段最值得投入时间的系统之一。

禁止：

`SetActorRotation()`

直接跟随输入。

高速状态应该存在明显的：

Turning Radius

近似：

```
TurnCapability =  
BaseTurnRate  
× Agility  
× SpeedFactor  
× TerrainFactor
```

速度越高：

Speed ↑
TurnRadius ↑

如果玩家强行高速急转：

增加：

BalanceRisk

达到阈值：

```
Normal  
↓  
Unstable  
↓  
Stumble  
↓  
Fall
```

9. 马的惯性

马应该具有明显的：

Momentum

释放W以后：

不是瞬间停止。

猛按反方向：

不是立即反向。

例如高速状态：

CurrentVelocity → 45 km/h North

DesiredDirection → West

运动系统应该逐步：

North



NorthWest



West

而不是：

North → West

这是马有没有“重量感”的关键。

10. 动画架构

建立：

ABP_Horse

第一阶段推荐：

Locomotion State Machine

Idle



Walk



Trot



Canter

↓

Gallop

同时增加：

Turn

Lean

Stop

Jump

Stumble

Rear

Kick

Struggle

后面最好逐渐走向：

Motion Matching / Pose Search

但是Prototype第一阶段不要为了先进动画系统拖慢核心玩法。

先把控制手感做正确。

11. Rider 系统

建立：

ARiderCharacter

以及：

URidingComponent

骑乘应该是明确状态：

OnFoot

Mounting

Mounted

Dismounting

Falling

Dragged

Mounted状态：

Player Controller仍然控制Rider。

Rider向Horse发送:

FRidingIntent

例如:

```
struct FRidingIntent
{
    float Forward;
    float Turn;
    bool bSprint;
    bool bBrake;
};
```

这是很重要的解耦。

12. Camera

建立:

URidingCameraComponent

不要用固定SpringArm完事。

至少考虑:

Speed
Acceleration
Turn
LassoAim
TerrainSlope
HorseState

例如:

Walk

FOV = 68
Distance = 350

Gallop

FOV ≈ 74
Distance ≈ 420

Sprint

FOV $\approx$ 78

Distance $\approx$ 460

不要把这些数值硬编码，它们只是Prototype起始值。

13. Camera Lag

一定需要：

轻微运动延迟。

目标是让玩家感到：

速度

而不是：

眩晕。

禁止用猛烈Camera Shake来制造速度。

速度感主要来自：

- 地面流动
- FOV
- 镜头距离
- 风声
- 马蹄
- -动画
- 植被掠过

14. 野马Brain

Prototype推荐：

StateTree + Utility Scoring

UE 当前已经把 StateTree 作为重要 AI/Gameplay Framework 继续演进，很适合把高层马行为写成清晰的状态结构。

状态：

Idle
Graze
Observe
Alert
MoveAway
Flee
ReturnToHerd
Exhausted
Lassoed
Struggle
Controlled

但:

什么状态应该被选择

不要完全依赖传统状态跳转。

由 Utility Score 决定。

15. Threat System

建立:

UHorseThreatComponent

Horse不断评估:

ThreatSource
Distance
ApproachSpeed
ApproachDirection
ThreatPersistence

计算:

ThreatScore

概念:

Threat =
DistanceFactor
× ApproachSpeedFactor
× Fearfulness
× DirectionFactor

例如：

玩家静止站在30米：

低威胁。

玩家骑马以50km/h正面冲来：

极高威胁。

这非常重要。

因为：

马会读行为，而不是只读距离。

16. Horse Perception

可以利用UE AI Perception做感知入口。

但不要让AI Perception直接决定行为。

应该：

AI Perception



Horse Perception



Threat Model



Utility Evaluation



Behavior

17. Herd Architecture

早期只有：

12 ~ 20匹马。

所以不要一开始直接MassEntity。

先用普通Actor保证：

玩法正确。

建立：

AHerdManager

每匹Horse：

UHerdComponent

保存：

HerdID

Leader

LocalPosition

DistanceFromCenter

PreferredSpacing

HerdAffinity

18. Herd Steering

至少实现：

Separation

避免拥挤。

Alignment

保持方向一致。

Cohesion

维持群体。

ThreatAvoidance

远离玩家。

TerrainAvoidance

避免石头、悬崖、树林过密区域。

最终：

DesiredDirection =
Separation
+ Alignment
+ Cohesion
+ ThreatAvoidance
+ TerrainAvoidance

每一项带权重。

19. Herd Pressure

这是《套马的汉子》第一个真正有原创潜力的Gameplay System。

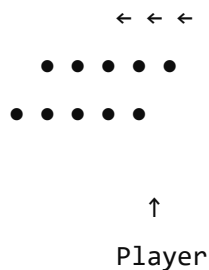
定义：

HerdPressure

玩家所在位置会产生：

Pressure Field

例如：



玩家从右下方接近：

马群整体往：

左上方移动。

这意味着玩家可以：

用自己的骑行路线塑造马群。

这是“赶马”玩法的基础。

20. Target Isolation

当目标马受到：

玩家压力

与：

马群吸引力

同时作用：

PlayerPressure

↓

TARGET

↑

HerdCohesion

高手需要让：

PlayerPressure > HerdCohesion

把目标从群体中切出来。

这应该成为套马之前的重要技能。

21. Lasso System

核心模块：

ULassoComponent

状态：

Stored

Equipped

Swinging

Thrown

Attached

Tensioned

Released

Broken

22. 一个重要决定

不要直接把 Unreal 的 Cable Component 当成全部套索Gameplay逻辑。

Cable可以承担：

绳子视觉表现的一部分。

但真正玩法逻辑使用自己的：

Rope Simulation Model

否则很难稳定控制：

- 套索长度
- 张力
- 网络同步
- Attachment
- 断裂
- Gameplay预测
- Debug

因此采用：

Gameplay Rope

+

Visual Rope

双层结构。

23. Lasso Throw

建立：

ALassoProjectile

但这里的Projectile不是普通子弹。

它描述：

LoopCenter

LoopRadius

Velocity

AngularVelocity

Expansion

Prototype可以采用简化判定：

目标马身上设置：

Head
Neck
Torso
FrontLeg
RearLeg

Lasso飞过对应Volume:

尝试捕获。

24. Rope Simulation

Prototype不要马上实现完整软体绳。

游戏逻辑层:

Anchor A = Rider / Horse

Anchor B = Wild Horse

CurrentLength

RestLength

MaximumLength

RelativeVelocity

计算:

Tension =
StretchForce
+
RelativeVelocityForce
+
TargetResistance

然后:

Tension / BreakingStrength

得到:

0.0 — Loose
0.4 — Loaded

0.7 — High
0.9 — Critical
1.0 — Break

25. 不要做“绳索血条”

玩家应该通过：

- 绳子拉直
- 绳索震动
- 手柄震动
- 绳索吱嘎
- Rider姿态
- Horse姿态
- 音效

判断风险。

HUD只提供辅助反馈。

26. Rope Gameplay

套中以后：

目标野马进入：

Lassoed

但仍拥有自己的AI。

例如：

EscapeForward

EscapeSide

Reverse

Circle

StopAndPull

Rear

马会尝试：

摆脱玩家。

所以Gameplay实际上变成：

Horse AI

VS

Player Rope Management

这才是游戏。

27. Target Resistance

建立：

UResistanceComponent

计算：

Resistance =

Strength

× Fear

× CurrentStamina

× Personality

于是：

同一套玩法面对不同马：

体验会不同。

28. Rider Balance

建立：

URiderBalanceComponent

变量：

Balance

SeatStability

PullDirection

RelativeVelocity

HorseAcceleration

出现：

Safe
Unstable
Critical
Fall

如果：

套中一匹力量特别大的马，

并且绳索：

90°侧向拉扯，

玩家极易：

落马。

29. Dragged State

这是Prototype必须保留的“喜剧制造器”。

角色落马后：

如果玩家没有Release：

Mounted

↓

Falling

↓

Dragged

玩家仍可以：

Release Lasso

否则：

继续被拖。

不要急着判断失败。

30. Capture状态

野马逐渐消耗：

Stamina
Fear Energy
Resistance

进入：

Flee
↓
Struggle
↓
Resist
↓
Exhausted
↓
Controlled

Controlled ≠ Tamed。

只是：

暂时停止剧烈反抗。

31. Trust

Prototype建立最小：

UTrustComponent

第一阶段只需要：

Trust = 0~100

但不要UI驱动。

当玩家：

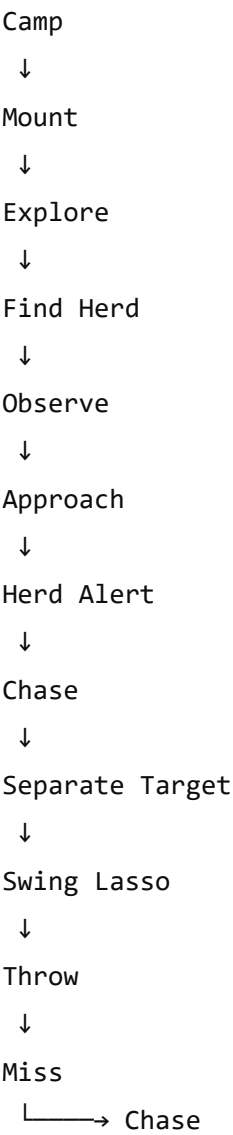
- 慢速靠近
- 不突然冲刺
- 正确控制绳索
- 抚摸
- 牵行

增加。

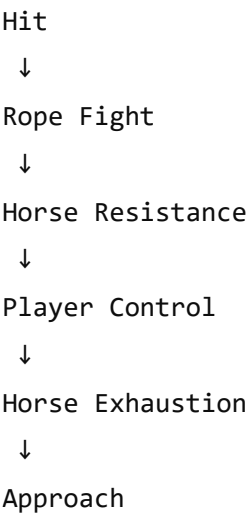
猛烈操作：

降低。

32. 最终Prototype循环



OR



↓
Lead
↓
Camp
↓
Name Horse

整个游戏Prototype:

就这一条链。

33. Gameplay Tags

强烈建议第一天开始使用 Gameplay Tags。

例如:

```
Horse.State.Idle  
Horse.State.Flee  
Horse.State.Lassoed  
Horse.State.Exhausted
```

```
Horse.Gait.Walk  
Horse.Gait.Trot  
Horse.Gait.Gallop
```

```
Rider.State.Mounted  
Rider.State.Falling  
Rider.State.Dragged
```

```
Lasso.State.Swinging  
Lasso.State.Thrown  
Lasso.State.Attached  
Lasso.State.Broken
```

后面系统会非常复杂。

Gameplay Tags会避免大量:

```
bool bIsRunning  
bool bIsLassoed  
bool bIsFalling  
bool bIsMounted  
...
```

互相冲突。

34. GAS现在要不要用？

我的建议：

Prototype暂时不要把整个项目建立在GAS上。

Gameplay Ability System当然非常强，Epic现在仍将它作为复杂Attributes、Abilities、Effects和多人同步的重要Gameplay框架。

但是套马Prototype最重要的是：

物理 + Movement + AI + Interaction

而不是技能系统。

所以：

现在：

不用GAS驱动核心Horse movement。

以后：

角色技能、骑术、特殊动作、Buff/Debuff、多人能力等出现后，再评估引入。

35. MassEntity什么时候使用？

现在：

不用。

后期：

玩家可能拥有：

300

500

1000+

只马。

这时不能全部使用完整：

ACharacter + SkeletalMesh + Brain

就可以形成：

Near Player

Full Horse Actor

Mid Distance

Simplified Actor

Far Distance

Mass Entity

Very Far

Abstract Herd Simulation

MassEntity正是UE的数据导向大规模实体框架，可以留作这一层扩展。

这个设计以后非常重要。

36. 世界技术路线

最终正式游戏：

World Partition

+

Landscape

+

PCG

+

Data Layers

+

HLOD

World Partition本身就是UE用于大地图自动分区、Streaming加载的系统，非常适合以后真正展开草原世界。

PCG则负责：

Grass

Bush

Rock

Forest

Water Edge

Animal Spawn Region

等程序化世界内容。

但：

Prototype地图不要开始做大世界。

37. Prototype Map

建立：

L_Prototype_Grassland

约：

2 km × 2 km

包含：

Flat Grassland

Hill

River

Sparse Forest

Rock Field

Camp

Simple Corral

目的完全是Gameplay测试。

不是景观作品。

38. Debug系统

从第一天做。

按：

F1 Horse Debug

F2 Herd Debug

F3 Threat Debug

F4 Rope Debug

F5 Movement Debug

例如Horse Debug:

Speed: 12.3 m/s

Stamina: 72%

Fear: 0.63

State: Flee

Desired Direction →

Actual Direction ↗

39. Herd Debug

显示:

Herd Center

Herd Direction

Leader

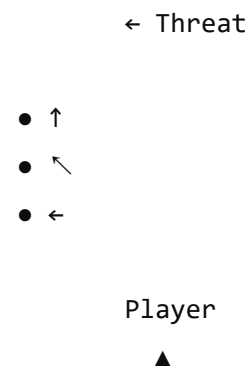
Pressure Vector

Cohesion Vector

Threat Vector

Final Steering Vector

最终你应该看到类似:



这会极大降低以后AI调试难度。

40. Rope Debug

显示:

Anchor A

Anchor B

Length

RestLength

RelativeVelocity

Tension

BreakThreshold

PullDirection

因为：

套索系统以后最容易出现：

“感觉不对，但是不知道哪里不对。”

Debug Visualization非常重要。

41. Telemetry

Prototype就记录：

AverageChaseTime

ThrowCount

HitCount

CaptureCount

RopeBreakCount

PlayerFallCount

DraggedDistance

TargetEscapeCount

以后真正测试时：

不是：

“感觉好像挺好。”

而是知道：

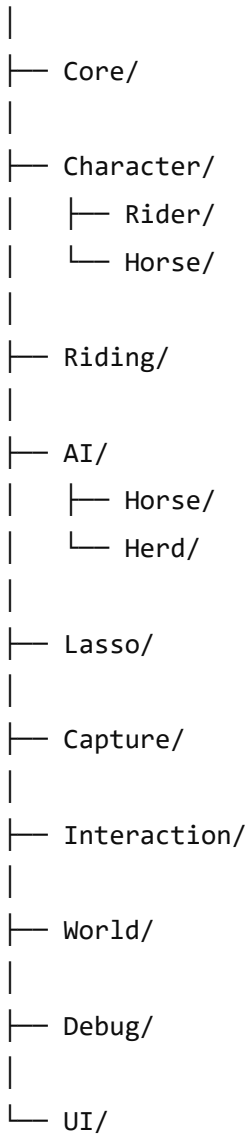
玩家平均：

3.4次抛索

成功一次。

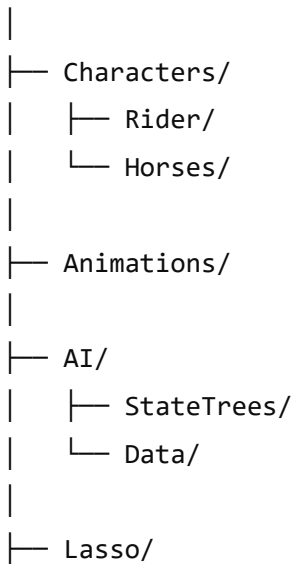
42. 文件目录建议

Source/Steppe/



Content:

Content/Steppe/




```
|
|— Worlds/
|   └─ Prototype/
|
|— Materials/
|
|— Audio/
|
|— FX/
|
└─ UI/
```

43. Data Driven

马的品种、身体属性、性格都不要硬编码。

例如：

UHorseArchetypeDataAsset

内容：

BaseSpeed

BaseStamina

BaseStrength

BaseAgility

FearRange

IndependenceRange

BodyMassRange

Prototype可以创建：

DA_Horse_Normal

DA_Horse_Fast

DA_Horse_Strong

DA_Horse_Nervous

测试：

不同马是否真的存在Gameplay区别。

44. Phase P0 —— Project Bootstrap

目标：

建立干净的UE C++项目。

完成：

Steppe.uproject

Source structure

GameMode

PlayerController

Input Mapping Context

Gameplay Tags

Debug subsystem

Prototype Map

验收：

项目无警告启动。

45. Phase P1 —— Horse Locomotion

只实现：

Horse

Camera

Input

Movement

Animation

不要：

AI。

不要：

套索。

不要：

马群。

验收标准非常严格：

骑马在空草原10分钟，本身已经有明显乐趣。

如果这一阶段不好玩：

禁止继续P2。

46. P1验收重点

测试：

Acceleration

有没有重量。

Turn

有没有马感。

Gallop

有没有速度感。

Brake

有没有惯性。

Camera

有没有速度，同时不晕。

Animation

动作和速度是否匹配。

47. Phase P2 —— Wild Horse

增加：

HorseBrain

Perception

Threat

Fear

Flee
Stamina

一个玩家：

追一匹马。

验收：

即使没有马群，追逐一匹马已经需要技巧。

48. Phase P3 —— Herd

增加：

12-20 Wild Horses

Separation
Alignment
Cohesion
ThreatAvoidance
HerdPressure

验收：

玩家能够通过自己所在位置真正控制马群的大致运动方向。

49. Phase P4 —— Isolation Gameplay

增加：

Target Selection

让玩家：

从20匹马里盯上一匹。

然后：

通过路线把目标从马群切出去。

验收：

高手比新手明显更快完成Isolation。

50. Phase P5 —— Lasso Throw

只做：

Equip

Swing

Aim

Throw

Attach

Miss

Retrieve

暂时没有复杂张力。

验收：

连续练习10分钟以后，玩家的命中率明显提高。

也就是说：

存在技术成长。

51. Phase P6 —— Rope Fight

加入：

Tension

Resistance

Break

RiderBalance

Fall

Drag

HorseStamina

这是整个Prototype最重要的一阶段。

目标：

套中只是战斗开始。

52. Phase P7 —— Capture

加入：

Exhaustion

Controlled

Approach

Trust

Lead Horse

Camp Return

Naming

Prototype核心循环正式闭环。

53. Phase P8 —— Vertical Slice Polish

才开始加入：

Grass

Wind

Dust

Horse Breath

Hoof Audio

Rope Audio

Basic Music

Better Animations

UI

Horse Card

最终得到：

10分钟左右的Playable Vertical Slice。

54. 暂时禁止开发列表

没有完成P7以前禁止开发：

Breeding

Genetics

Economy

NPC Quests

Story

Crafting
Building
Multiple Biomes
Winter
Wolf AI
Multiplayer
Skill Tree
Large Open World
Hundreds of Horses
Complex Character Customization

原因不是这些功能不好。

而是：

它们会掩盖核心玩法是否真的成立。

55. 第一阶段真正的技术风险

按优先级：

RISK 1

Horse locomotion不像动物。

RISK 2

追逐只是跟在屁股后面跑。

RISK 3

马群AI看起来像粒子。

RISK 4

套索像射击武器。

RISK 5

Rope Physics不可控。

RISK 6

套中以后结果过于随机。

RISK 7

玩家无法理解失败原因。

其中：

前三项甚至比画质重要得多。

56. 核心设计法则

以后所有Cursor/Codex生成的代码都遵守：

Rule 1

AI决定意图，Movement决定结果。

Rule 2

输入决定请求，不直接决定马的位置。

Rule 3

套索Gameplay Simulation和Visual Rope解耦。

Rule 4

Horse个体属性必须进入实际Gameplay计算。

Rule 5

马群是系统，不是20个独立AI碰巧站在一起。

Rule 6

失败必须可以理解。

Rule 7

让系统制造故事。

最后这一条尤其重要。

不是提前编：

“玩家被马拖进河里。”

而是建立物理和AI规则，

让：

“玩家真的可能被马拖进河里。”

57. Prototype真正完成的标志

不是：

所有Feature Checkbox = Done

而是某一天内部测试的时候：

有人准备只试10分钟，

最后玩了：

40分钟。

一直在说：

“等一下，我再抓一匹。”

到这里，

《套马的汉子》才真正值得继续做下去。

这里有几个技术决策我认为现在就应该定死：

第一，不要把马做成“有马皮肤的载具”。 UE 的 Character、Animation、StateTree 都只是工具，真正核心仍然应该是我们自己的 HorseIntent → HorseMovement 层。

第二，暂时不要上 MassEntity。 MassEntity很适合以后处理数百甚至上千个远距离个体，但 Prototype 的 20 匹野马应该全部是完整 Actor，因为现在优化的对象不是性能，而是行为。等以后真正出现几百匹马，再做 Full Actor → Simplified Actor → Mass Entity → Abstract Herd 四级模拟。  Epic Games Dev...

第三，套索不要完全依赖 Chaos/Cable 的实时物理。 Chaos负责物理世界当然很合适，但“套索长度、张力、断裂、玩家平衡、马匹反抗”必须有一层确定性的 Gameplay Solver。否则很可能做出一个技术上很真实、玩起来却不可控的绳子。

第四，我甚至不建议第一阶段先找一匹特别精致的马素材。 先用质量尚可、动画齐全的临时马，把 P1 Horse Locomotion 做出来。真正危险的是大家被漂亮草原、Lumen、Nanite、马毛和夕阳吸引，三个月以后才发现“骑马不好玩”。

而且从这里开始，我会把项目开发顺序固定成：

P0 工程骨架 → P1 骑马 → P2 追一匹马 → P3 追一个马群 → P4 切马 → P5 抛索 → P6 拉扯 → P7 捕获 → P8 视觉切片。

下一次继续推进时，就已经不用再讨论宏观概念了，可以直接进入 **P0 + P1 的工程级设计**，包括具体 UE C++ 类、.h/.cpp 职责、Enhanced Input、Horse Movement 数学模型、AnimBP 参数、摄像机算法、Debug HUD，以及一份可以直接交给 Cursor/Codex 开始创建 Unreal 项目的 **Master Prompt**。